6G Digital Twin Network - Phase 1 Implementation

Project Overview

Implementation of a Digital Twin Network (DTN) for network simulation, monitoring, and optimization based on the IEEE Communications Magazine paper "6G Digital Twin Networks: From Theory to Practice".

Current Status: Phase 1 - Core Infrastructure

Phase 1 Components

1. **Physical Network Layer** - Mininet topology simulation
2. **Data Collection Pipeline** - Structured storage with SQLite
3. **Digital Twin Core** - Real-time synchronization loop
4. **Live Dashboard** - Flask + Plotly visualization
5. **Custom Topology Builder** - YAML-based configuration

Project Structure

```
dtn-project/
├── config/
│   ├── topology_config.yaml      # Network topology definitions
│   └── collection_config.yaml    # Data collection settings
├── physical_network/
│   ├── mininet_topology.py       # Mininet network setup
│   └── topology_builder.py       # Dynamic topology from YAML
├── data_layer/
│   ├── collector.py               # Data collection engine
│   ├── storage.py                 # SQLite database interface
│   └── schema.sql                 # Database schema
├── twin_core/
│   ├── sync_engine.py             # Real-time sync loop
│   └── state_manager.py           # Twin state management
├── dashboard/
│   ├── app.py                     # Flask application
│   ├── templates/
│   │   └── index.html             # Dashboard UI
│   └── static/
│       └── css/
│           └── style.css          # Dashboard styling
├── tests/
│   └── test_collection.py         # Unit tests
```

```
├── requirements.txt                # Python dependencies
└── README.md                       # This file
```

Installation

Prerequisites

- Python 3.8+
- Mininet (for network simulation)
- SQLite3

Setup

```bash
# Install Python dependencies
pip install -r requirements.txt

# Install Mininet (Ubuntu/Debian)
sudo apt-get install mininet

# Initialize database
python data_layer/storage.py --init
```

Quick Start

1. Define Your Topology

Edit `config/topology_config.yaml`:

```yaml
network:
  name: "sample_network"
  nodes:
    - name: "h1"
      type: "host"
    - name: "s1"
      type: "switch"
  links:
    - src: "h1"
      dst: "s1"
      bandwidth: 100  # Mbps
      delay: "10ms"
```

2. Start the Physical Network

```bash
sudo python physical_network/topology_builder.py --config config/topology_config.yam
```

3. Launch Data Collection

```bash
python data_layer/collector.py --interval 1
```

4. Start the Dashboard

```bash
python dashboard/app.py
```

Access at: http://localhost:5000

Data Schema

Network Metrics Table

```sql
CREATE TABLE network_metrics (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
    node_src TEXT NOT NULL,
    node_dst TEXT NOT NULL,
    latency_ms REAL,
    throughput_mbps REAL,
    packet_loss_pct REAL,
    jitter_ms REAL
);
```

Phase 1 Timeline

- **Days 1-2:** Physical network + topology builder
- **Day 3:** Data collection + structured storage
- **Day 4:** Digital twin core + sync engine
- **Days 5-6:** Dashboard development
- **Day 7:** Integration testing and validation

Next Steps (Phase 2)

- Rule-based congestion detection
- Dijkstra route optimization
- OpenFlow controller integration
- Closed-loop control implementation

References

- Paper: "6G Digital Twin Networks: From Theory to Practice" (IEEE Communications Magazine, Nov 2023)
- ITU-T Recommendation Y.3090: Digital Twin Network Requirements and Architecture